

Wybrane zagadnienia Algebry

Rafał Wochna 279752

1 Zad1 Pierścień liczb Gaussa $\mathbb{Z}[i]$

$$a = 2, \quad b = 7, \quad c = 9, \quad d = 7, \quad e = 5, \quad f = 2.$$

Liczbę $z = x + yi$ reprezentuję jako parę liczb całkowitych (x, y) , gdzie x to część rzeczywista, y to część urojona. Klasa `GaussianInteger` przechowuje te wartości jako pola `real` i `imag`. Normę liczby obliczam ze wzoru $N(a + bi) = a^2 + b^2$:

Dzielenie wykonuję przez zaokrąglenie ilorazu rzeczywistego do najbliższej liczby Gaussa:

$$\frac{z_1}{z_2} = \frac{(x_1 + iy_1)(x_2 - iy_2)}{x_2^2 + y_2^2} = \frac{(x_1x_2 + y_1y_2) + i(y_1x_2 - x_1y_2)}{x_2^2 + y_2^2}.$$

Następnie zaokrąglam część rzeczywistą i urojoną do najbliższych liczb całkowitych.

```
1 int numReal = real * divisor.real + imag * divisor.imag;  
2 int numImag = imag * divisor.real - real * divisor.imag;  
3 int quotientReal = round((double)numReal / denominator);  
4 int quotientImag = round((double)numImag / denominator);
```

Listing 1: Dzielenie z resztą – fragment

NWD realizuję za pomocą algorytmu Euklidesa dla liczb Gaussa.

```
1 GaussianInteger x = a;  
2 GaussianInteger y = b;  
3 while (y.norm() != 0) {  
4     auto [quotient, remainder] = x.divideWithRemainder(y);  
5     x = y;  
6     y = remainder;  
7 }
```

Listing 2: Algorytm Euklidesa dla liczb Gaussa (NWD)

NWW obliczam ze wzoru:

$$\text{NWW}(a, b) = \frac{a \cdot b}{\text{NWD}(a, b)}.$$

Dla listy liczb implementuję funkcje `gcdListAll` i `lcmListAll`, które działają rekurencyjnie.

Wynik normy dla liczby $a + bi = 2 + 7i$:

$$N(2 + 7i) = 2^2 + 7^2 = 4 + 49 = 53.$$

Dzielenie $(c + a) + (d + b)i = (9 + 2) + (7 + 7)i = 11 + 14i$ przez $e + fi = 5 + 2i$.

Otrzymane wyniki (wszystkie możliwe ilorazy z resztą o normie mniejszej od dzielnika):

Iloraz	Reszta	$N(\text{reszta})$
$2 + 2i$	5	25
$3 + i$	$-2 + 3i$	13
$3 + 2i$	$-2i$	4

Uzasadnienie: Liczby Gaussa tworzą na płaszczyźnie siatkę kwadratów jednostkowych. Ponieważ odległość (norma) punktu wewnątrz kwadratu od jego wierzchołków może być mniejsza niż 1 dla nawet 4 z nich naraz, dzielenie dopuszcza kilka poprawnych reszt. W tym konkretnym przypadku czwarty wierzchołek kwadratu odpada, ponieważ daje resztę o zbyt dużej normie ($34 \geq 29$)

Wyniki dla trójki liczb: $2 + 7i$, $9 + 7i$, $5 + 7i$:

NWD : $(-1, -i, i, 1)$ NWW : $(-431 - 263i, -263 + 431i, 263 - 431i, 431 + 263i)$ Lista pusta: NWD = 0, NWW = 1 Lista 1-elementowa $(2 + 7i)$:

$$\text{NWD} = \{-7 + 2i, -2 - 7i, 2 + 7i, 7 - 2i\}$$

$$\text{NWW} = \{-7 + 2i, -2 - 7i, 2 + 7i, 7 - 2i\}$$

2 Zad2 Pierścień wielomianów $R[x]$

Wielomian reprezentuję jako wektor współczynników od wyrazu wolnego do najwyższej potęgi:

$$p(x) = c_0 + c_1x + c_2x^2 + \dots + c_nx^n \quad \leftrightarrow \quad \text{vector}[0..n] = [c_0, c_1, \dots, c_n]$$

Taka reprezentacja ułatwia dostęp do współczynnika przy x^k poprzez indeks k .

W pierścieniu $R[x]$ normą (stopniem) wielomianu jest najwyższa potęga x z niezerowym współczynnikiem.

Algorytm realizuje "dzielenie pisemne" wielomianów:

```

1 while (remainder.degree() >= divisorDeg && !remainder.isZero()) {
2     int degDiff = remainder.degree() - divisorDeg;
3     double factor = remainder.coeff.back() / leadingCoeff;
4     // tworzenie term = factor * x^{degDiff}
5     quotient = quotient + term;
6     remainder = remainder - (term * divisor);
7 }

```

Listing 3: Dzielenie wielomianów

Implementuję rozszerzony algorytm Euklidesa zwracający $g(x), s(x), t(x)$ takie że:

$$s(x) \cdot v(x) + t(x) \cdot w(x) = g(x) = \text{NWD}(v, w)$$

Algorytm działa w sposób rekurencyjny. W każdym kroku wykonuje dzielenie z resztą i schodzi w głąb dla pary (dzielnik, reszta).

```

1 static tuple<Polynomial, Polynomial, Polynomial> extendedGCD(
2     const Polynomial& a, const Polynomial& b) {
3     if (b.isZero()) return {a.normalized(), Polynomial({1.0}),
4         Polynomial({0.0})};
5     auto [q, r] = a.divide(b);
6     auto [g, s1, t1] = extendedGCD(b, r);
7     return {g, t1, s1 - t1 * q};
8 }

```

Listing 4: Rozszerzony NWD – rekurencyjnie

Wielomian $c \cdot x^a + b = 9x^2 + 7$ norma:

$$N(9x^2 + 7) = 2$$

Dzielenie:

$$\frac{9x^2 + 7}{x + 1}$$

Wynik:

$$\text{Iloraz} = 9x - 9, \quad \text{Reszta} = 16$$

Sprawdzenie: $(9x - 9)(x + 1) + 16 = 9x^2 + 9x - 9x - 9 + 16 = 9x^2 + 7$.

Rozszerzony NWD dla $v(x)$ i $w(x)$:

$$v(x) = ax^3 + bx^2 + cx + d = 2x^3 + 7x^2 + 9x + 7$$

$$w(x) = dx^3 + ex^2 + fx = 7x^3 + 5x^2 + 2x$$

$$\text{NWD}(v, w) = 1$$

$$s(x) \approx 0.0594043x^2 - 0.208679x + 0.142857$$

$$t(x) \approx -0.0169727x^2 + 0.0123416x + 0.0875197$$

Sprawdzenie: $s \cdot v + t \cdot w = 1$.

Na podstawie rozszerzonego algorytmu Euklidesa oraz wolframa Alpha wyznaczono stałą g , dla której $1 \notin \text{NWD}(v(x), w(x) + g)$:

$$g \approx 52.2738407$$

Dla tej wartości wielomiany $v(x)$ i $w(x) + g$ nie są względnie pierwsze.

$$w(x) + g \approx 7x^3 + 5x^2 + 2x + 52.2738407$$

$$\text{NWD}(v(x), w(x) + g) \approx x + 2.16937$$

$$\text{NWW} \approx 14x^5 + 28.6288x^4 + 39.8935x^3 + 126.004x^2 + 145.568x + 168.674$$

3 Zad3 Porządki produktowe

Dla $n \in \mathbb{N}$ definiujemy porządek produktowy $\leq$ na $\mathbb{N}^n$ jako:

$$(x_1, \dots, x_n) \leq (y_1, \dots, y_n) \iff \forall i \in \{1, \dots, n\} : x_i \leq y_i.$$

Dla skończonego zbioru $A \subset \mathbb{N}^n$, element $p \in A$ jest minimalny, jeśli nie istnieje $q \in A$, $q \neq p$, taki że $q \leq p$. Algorytm porównuje każdy punkt $p \in A$ ze wszystkimi pozostałymi punktami $q \in A$. Jeśli znajdzie jakiegokolwiek punkt q , dla którego zachodzi $q \leq p$, odrzuca p jako nieminimalny. W przeciwnym razie dodaje go do listy wyników.

```
1 template<typename T>
2 vector<T> findMinimals(const vector<T>& points) {
3     vector<T> result;
4     for (size_t i = 0; i < points.size(); ++i) {
5         bool isMinimal = true;
6         for (size_t j = 0; j < points.size(); ++j) {
```

```

7         if (i != j && leq(points[j], points[i])) {
8             isMinimal = false;
9             break;
10        }
11    }
12    if (isMinimal) result.push_back(points[i]);
13 }
14 return result;
15 }

```

Listing 5: Algorytm findMinimals

Złożoność: $O(|A|^2 \cdot n)$.

a) – pary w $\mathbb{N}^2$ Zbiór: $\{(2, 7), (9, 7), (5, 2)\}$. **Elementy minimalne:** $(2, 7)$ i $(5, 2)$.

b) – trójki w $\mathbb{N}^3$ Zbiór: $\{(2, 9, 5), (7, 7, 2)\}$. **Elementy minimalne:** $(2, 9, 5)$ i $(7, 7, 2)$

c) – zbiór $A = \{(x, y) \in \mathbb{N}^2 : (x - 2)^2 + (y - 7)^2 < 5\}$

Rozwiązując nierówność:

$$(x - 2)^2 + (y - 7)^2 \leq 4$$

Otrzymujemy 13 punktów dla $\mathbb{N}$ z zerem, 12 dla $\mathbb{N}$ bez zera.

$$\frac{\mathbb{N} = \{0, 1, 2, \dots\}}{(0, 7), (1, 6), (2, 5)} \mid \frac{\mathbb{N} = \{1, 2, 3, \dots\}}{(1, 6), (2, 5)}$$

d) – zbiór $B \subset \mathbb{N}^4$

$$B = \left\{ (x_1, x_2, x_3, x_4) \in \mathbb{N}^4 : \sum_{i=1}^4 (x_i - c_i)^2 > 224 \right\},$$

gdzie $(c_1, c_2, c_3, c_4) = (9, 7, 5, 2)$.

Zbiór B jest nieskończony, ale elementy minimalne leżą w ograniczonym obszarze. Dla każdej współrzędnej x_i rozważam tylko dwie możliwości:

1. $x_i = 0$ – ponieważ zero minimalizuje współrzędną,
2. $x_i \geq c_i + 1$ – aby mieć szansę spełnić warunek > 224 .

Maksymalna wartość współrzędnej jest ograniczona przez $c_i + 15$, ponieważ $15^2 = 225 > 224$.

Znaleziono **43 elementów minimalnych**

Elementy minimalne zbioru B			
(0, 0, 0, 11)	(0, 16, 0, 9)	(19, 0, 13, 6)	(20, 0, 11, 7)
(0, 0, 11, 10)	(0, 16, 11, 8)	(19, 0, 14, 0)	(20, 0, 12, 5)
(0, 0, 12, 9)	(0, 16, 12, 6)	(19, 15, 0, 8)	(20, 0, 13, 0)
(0, 0, 13, 8)	(0, 16, 13, 0)	(19, 15, 11, 7)	(20, 15, 0, 6)
(0, 0, 14, 6)	(0, 17, 0, 7)	(19, 15, 12, 6)	(20, 15, 11, 0)
(0, 0, 15, 0)	(0, 17, 11, 5)	(19, 15, 13, 0)	(20, 16, 0, 0)
(0, 15, 0, 10)	(0, 17, 12, 0)	(19, 16, 0, 7)	(21, 0, 0, 5)
(0, 15, 11, 9)	(0, 18, 0, 0)	(19, 16, 11, 5)	(21, 0, 11, 0)
(0, 15, 12, 8)	(19, 0, 0, 10)	(19, 16, 12, 0)	(21, 15, 0, 0)
(0, 15, 13, 6)	(19, 0, 11, 9)	(19, 17, 0, 0)	(22, 0, 0, 0)
(0, 15, 14, 0)	(19, 0, 12, 8)	(20, 0, 0, 8)	